

Tarea 3: Polars vs Pandas

Predicción de retrasos de vuelos

Curso: Computación Paralela

Profesor: Johansell Villalobos Cubillo

Estudiante: Carolina Salas

Institución: LEAD University

Fecha: Junio 2026

Informe experimental reproducible

1. Problema, dataset y metodología

El objetivo es predecir la variable binaria Delay y comparar el rendimiento de dos implementaciones equivalentes del procesamiento: Polars y Pandas. El dataset público de Kaggle contiene 539,383 vuelos y 9 variables. La clase 0 reúne 299,119 registros y la clase 1, 240,264.

Variables

Variable	Tipo	Descripción
id	Numérica	Identificador del registro
Airline	Categórica	Código de aerolínea
Flight	Numérica	Número de vuelo
AirportFrom / AirportTo	Categórica	Aeropuertos de origen y destino
DayOfWeek	Discreta	Día de la semana, de 1 a 7
Time	Numérica	Minutos desde medianoche
Length	Numérica	Duración programada
Delay	Binaria	Variable objetivo

Pipeline

El pipeline aplica validación de rangos, imputación defensiva, filtrado vectorizado, creación de Hour e IsWeekend, escalado, binning de duración, encoding determinista, group_by y join con frecuencia por aerolínea. La misma lógica se implementa en Pandas para mantener una comparación válida.

Entorno experimental

Windows-10-10.0.26200-SP0; 10 núcleos físicos y 16 lógicos; 31.63 GB RAM; dataset de 17.66 MB; Python 3.11.7; Polars 1.44.1; Pandas 3.0.5. Cada benchmark usa 5 repeticiones.

2. Resultados experimentales

Tiempos por operación

Operación	Polars (s)	Pandas (s)	Speedup
Read	0.0168	0.4875	29.03x
Filter	0.0067	0.0159	2.39x
Aggregation	0.0032	0.0149	4.61x
Join	0.0097	0.0792	8.17x
FeatureEngineering	0.0629	0.4664	7.41x
TotalPipeline	0.0759	1.0039	13.23x

El pipeline total tardó 0.0759 s en Polars y 1.0039 s en Pandas, con una aceleración de 13.23x. Polars fue más rápido en las seis operaciones.

Escalabilidad

Porcentaje	Filas	Polars (s)	Pandas (s)	Speedup
25%	134,845	0.0219	0.1186	5.42x
50%	269,691	0.0481	0.2398	4.98x
75%	404,537	0.0499	0.3453	6.92x
100%	539,383	0.0562	0.5240	9.32x

Lazy Execution

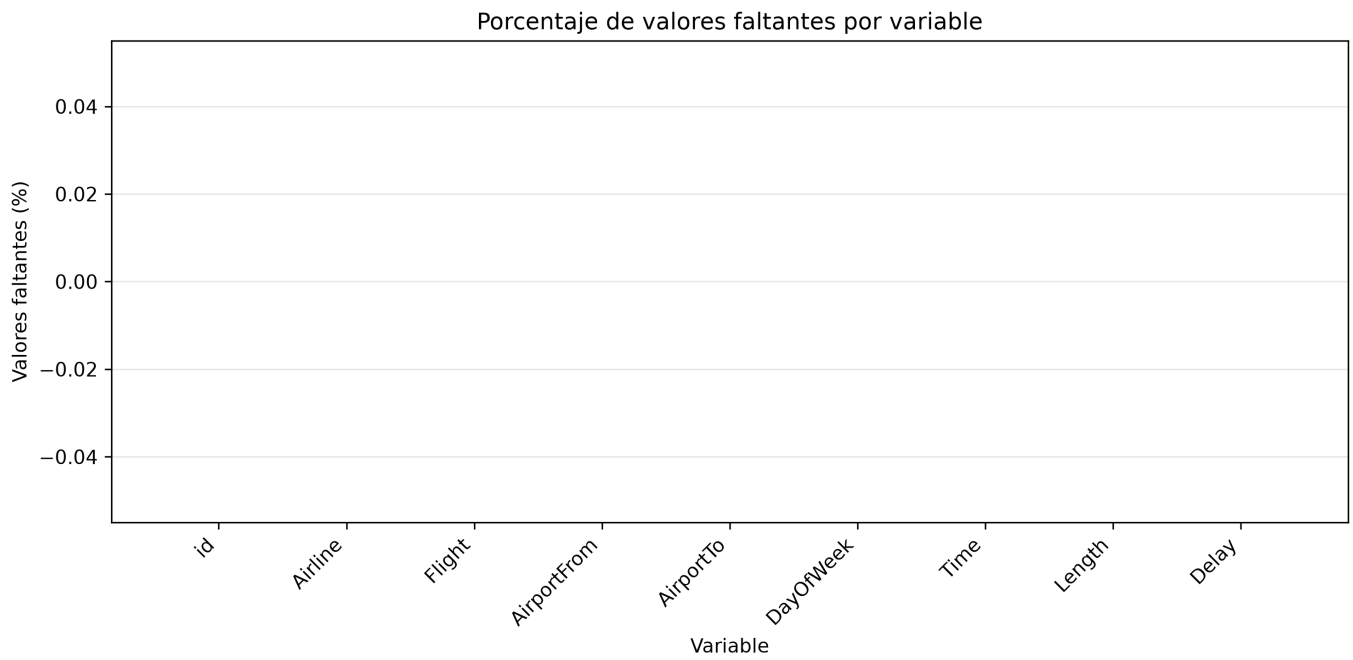
read_csv tardó 0.0250 s y aumentó el pico de memoria 20.69 MB; scan_csv().collect() tardó 0.0225 s y aumentó el pico 24.08 MB. Lazy fue ligeramente más rápido, pero consumió más memoria en este flujo corto.

Modelos predictivos

Modelo	Accuracy	F1 weighted	AUC	Tiempo (s)
Logistic Regression	0.6256	0.6159	0.6563	0.5840
Random Forest	0.6170	0.6164	0.6576	9.0964
XGBoost	0.6620	0.6512	0.7134	2.9642

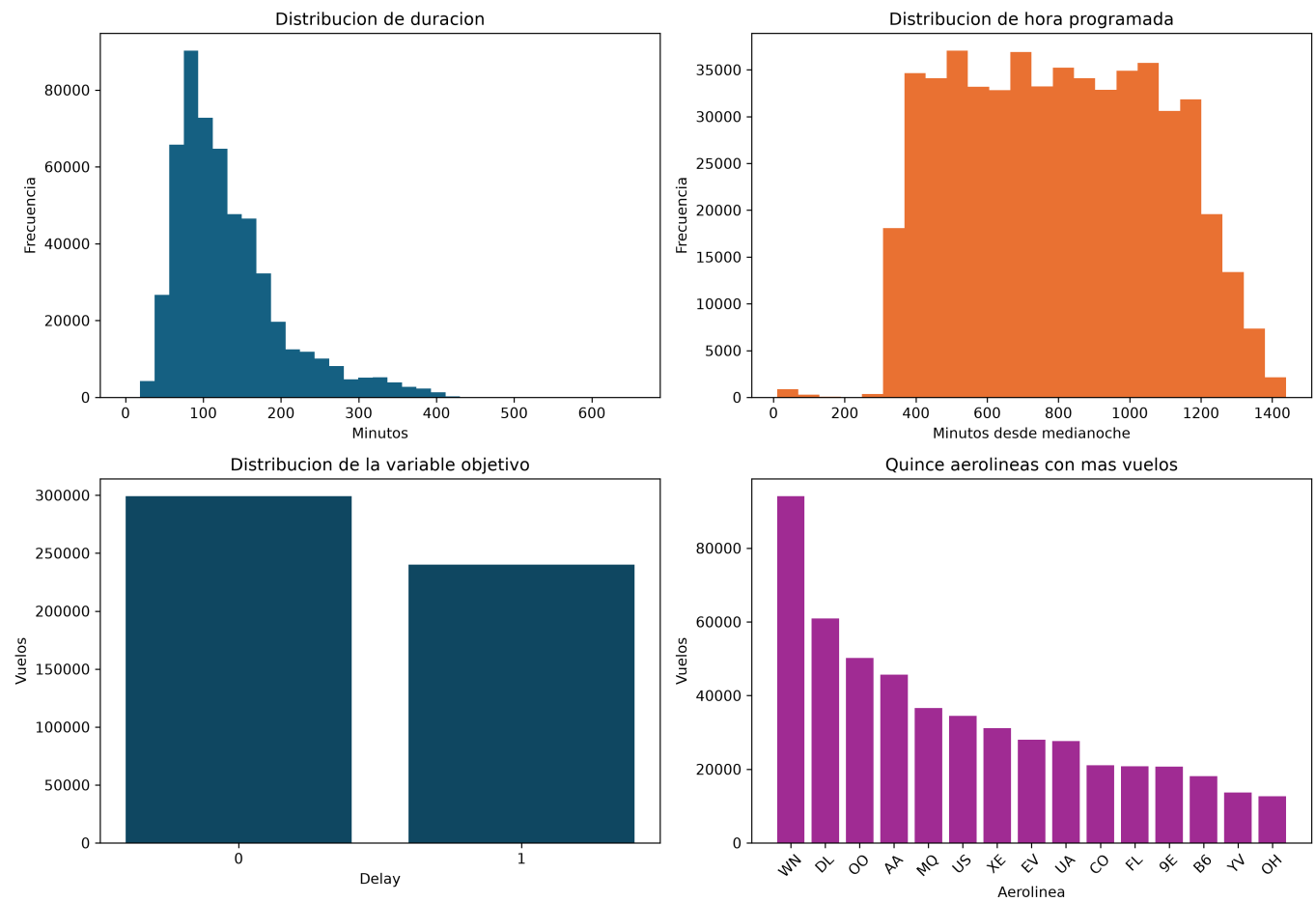
XGBoost obtuvo el mejor resultado: Accuracy 0.6620, F1 weighted 0.6512 y AUC 0.7134.

3. Valores faltantes



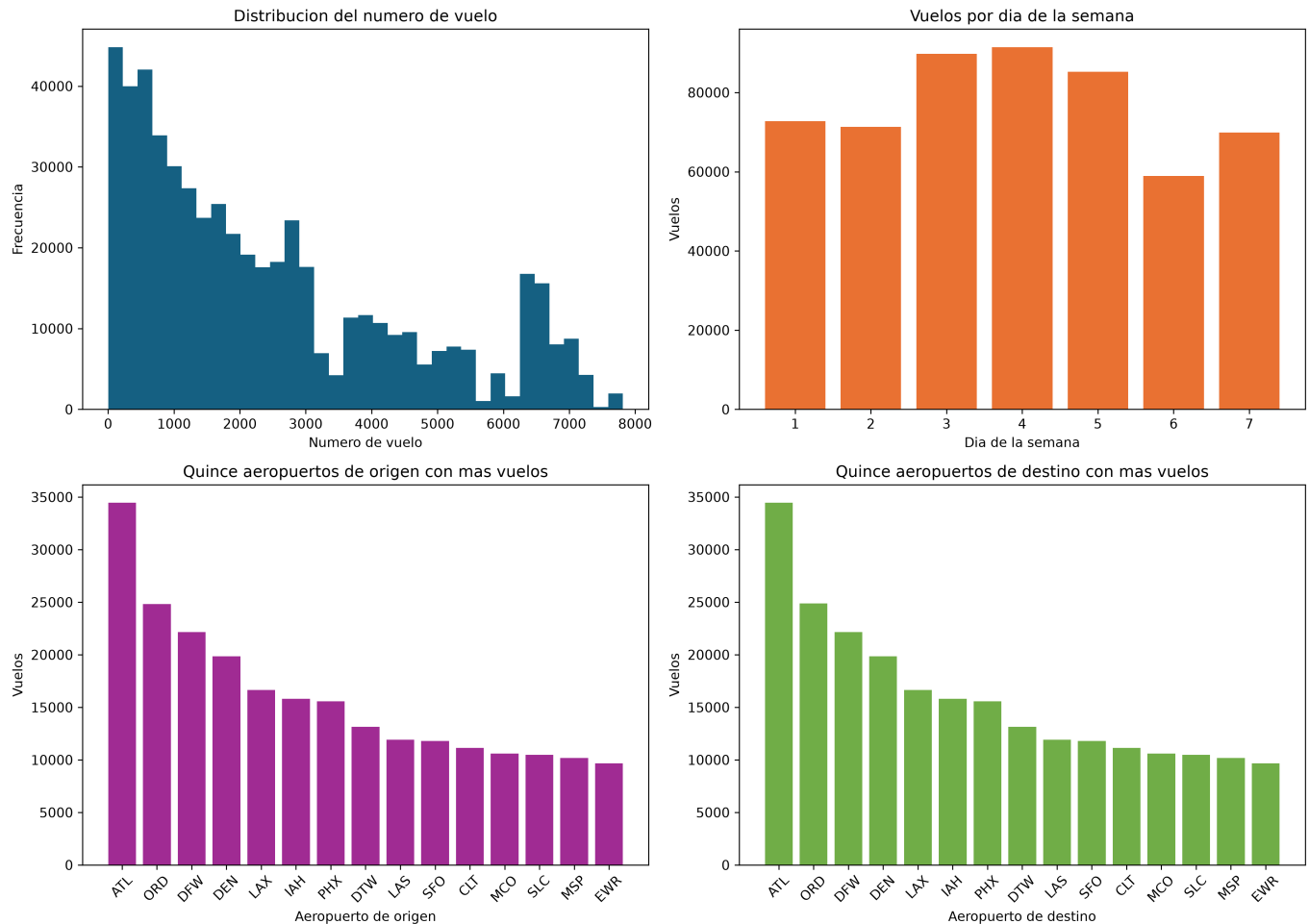
Las nueve variables tienen cero valores faltantes. Se conserva imputación defensiva para asegurar la reutilización del pipeline.

4. Distribuciones principales



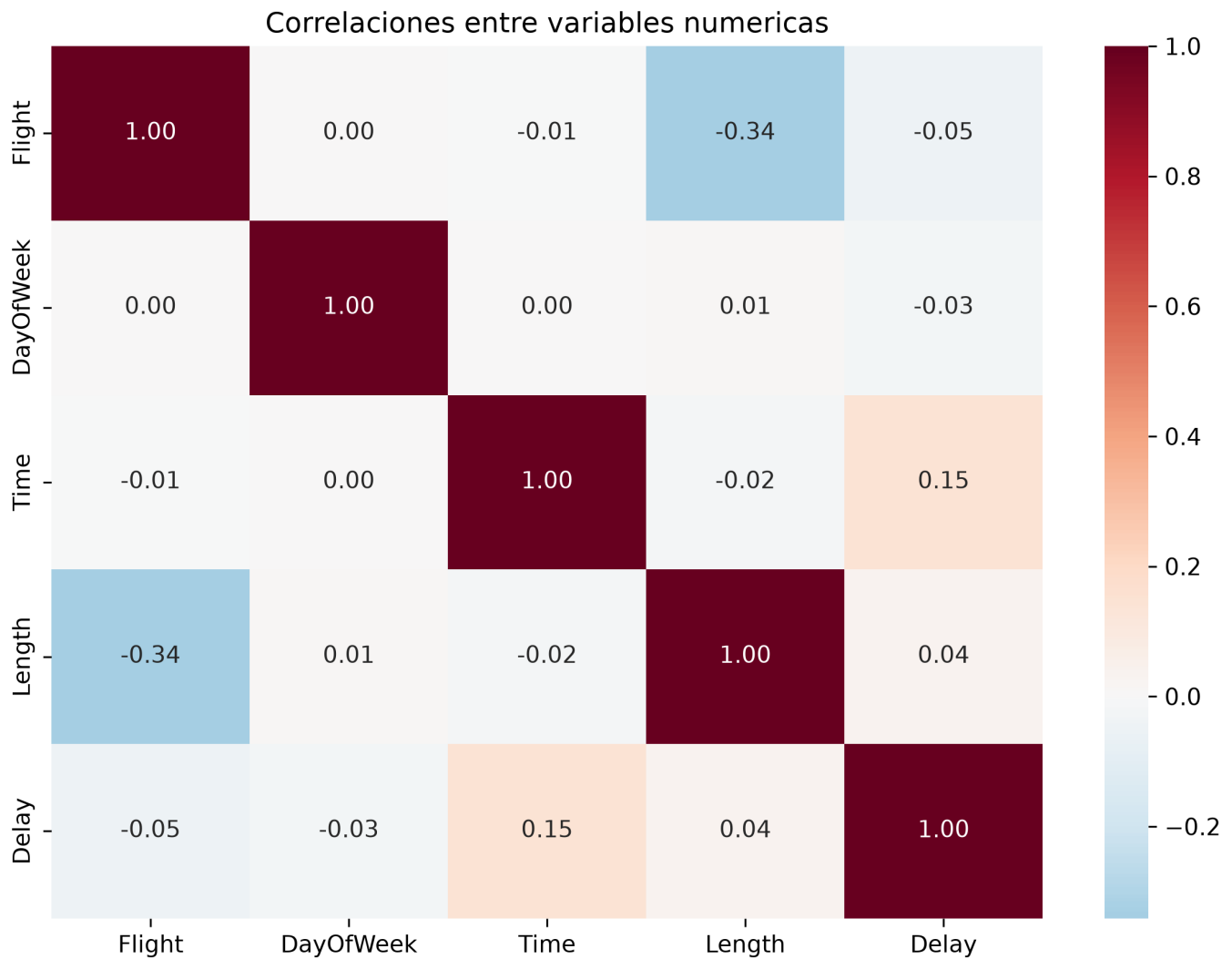
Distribuciones etiquetadas de Time, Length y Delay obtenidas después de la exploración con Polars.

5. Distribuciones adicionales



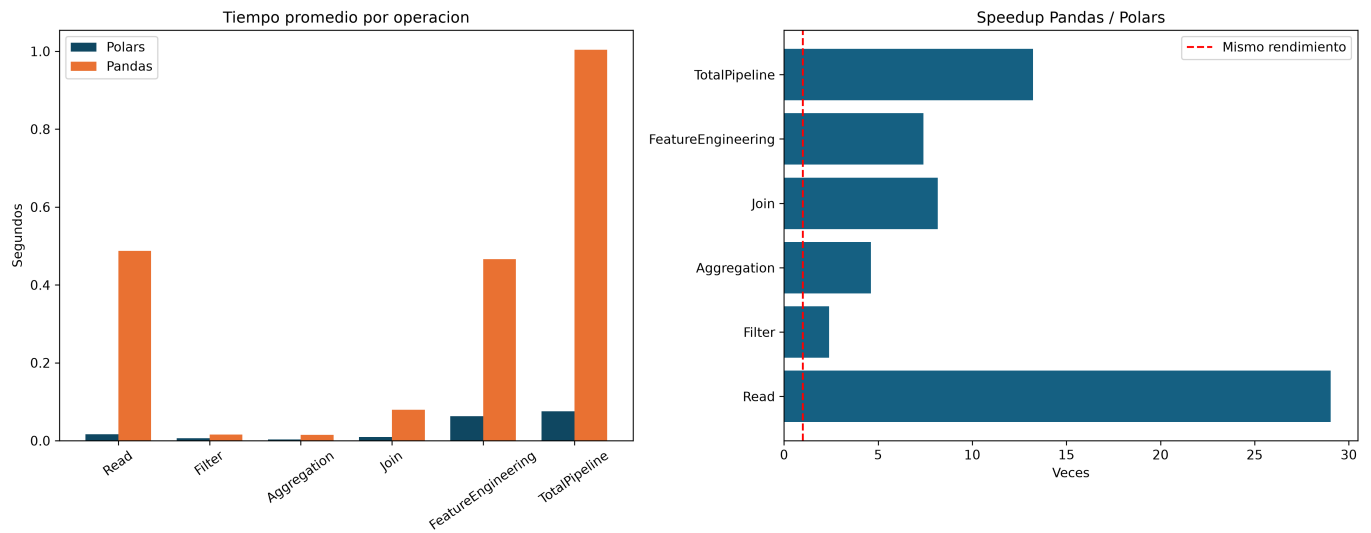
Análisis de Flight, DayOfWeek y categorías principales de origen y destino.

6. Correlaciones



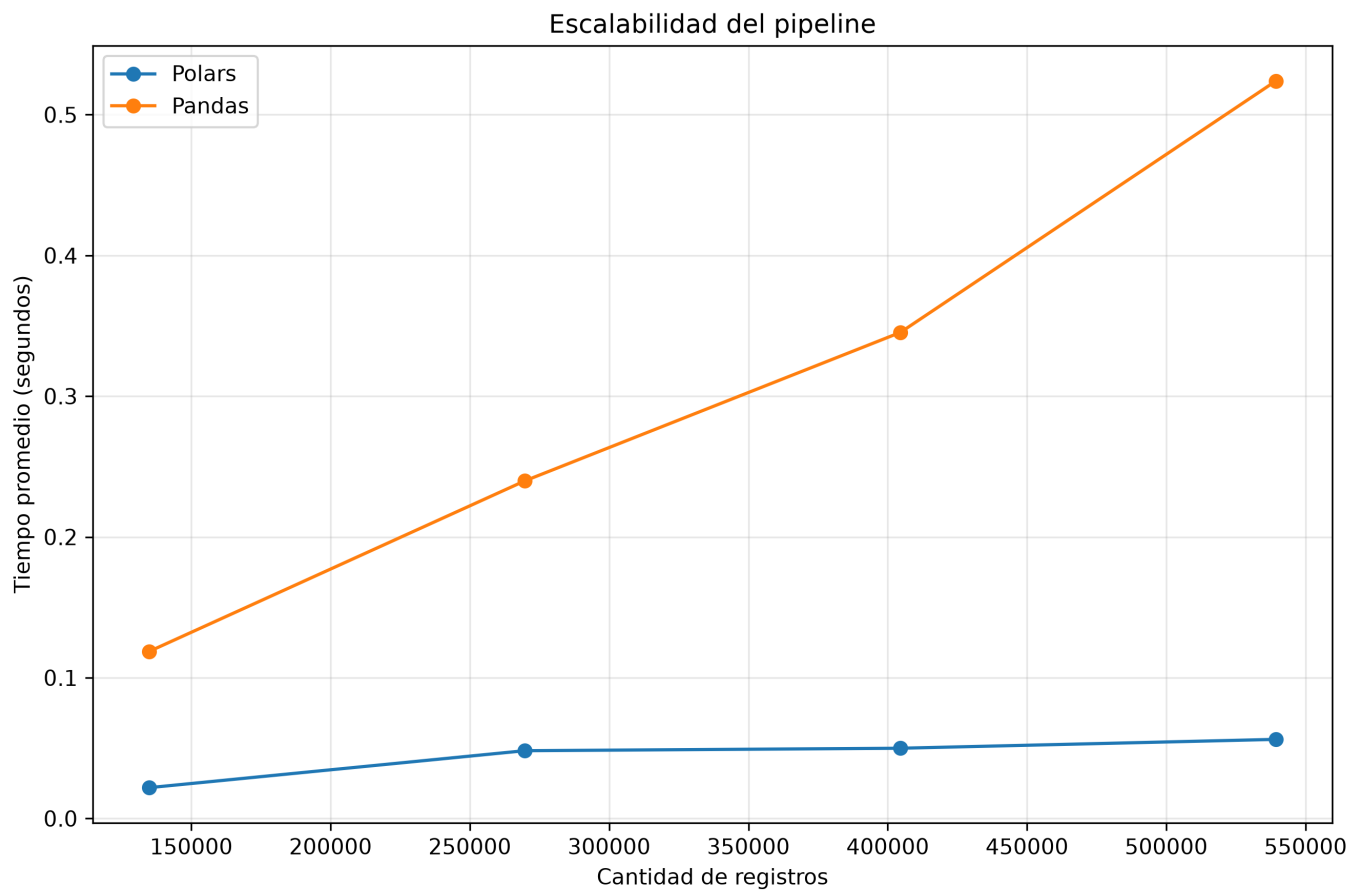
Matriz de correlaciones para variables numéricas; la interpretación evita asumir causalidad.

7. Benchmark y speedup



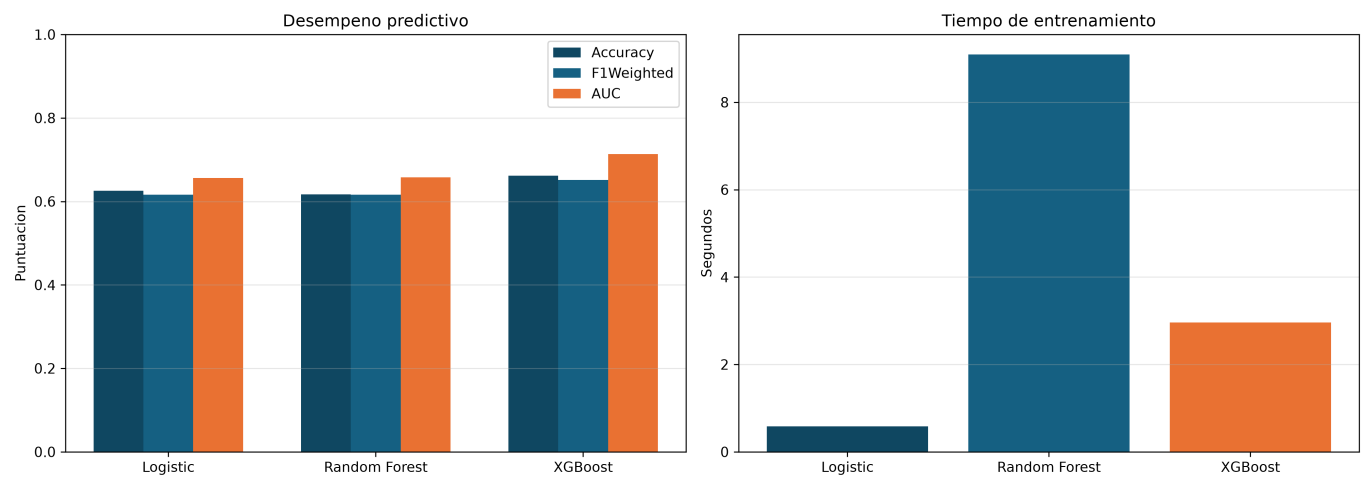
Comparación de tiempos promedio y speedup Pandas/Polars por operación y para el pipeline completo.

8. Escalabilidad



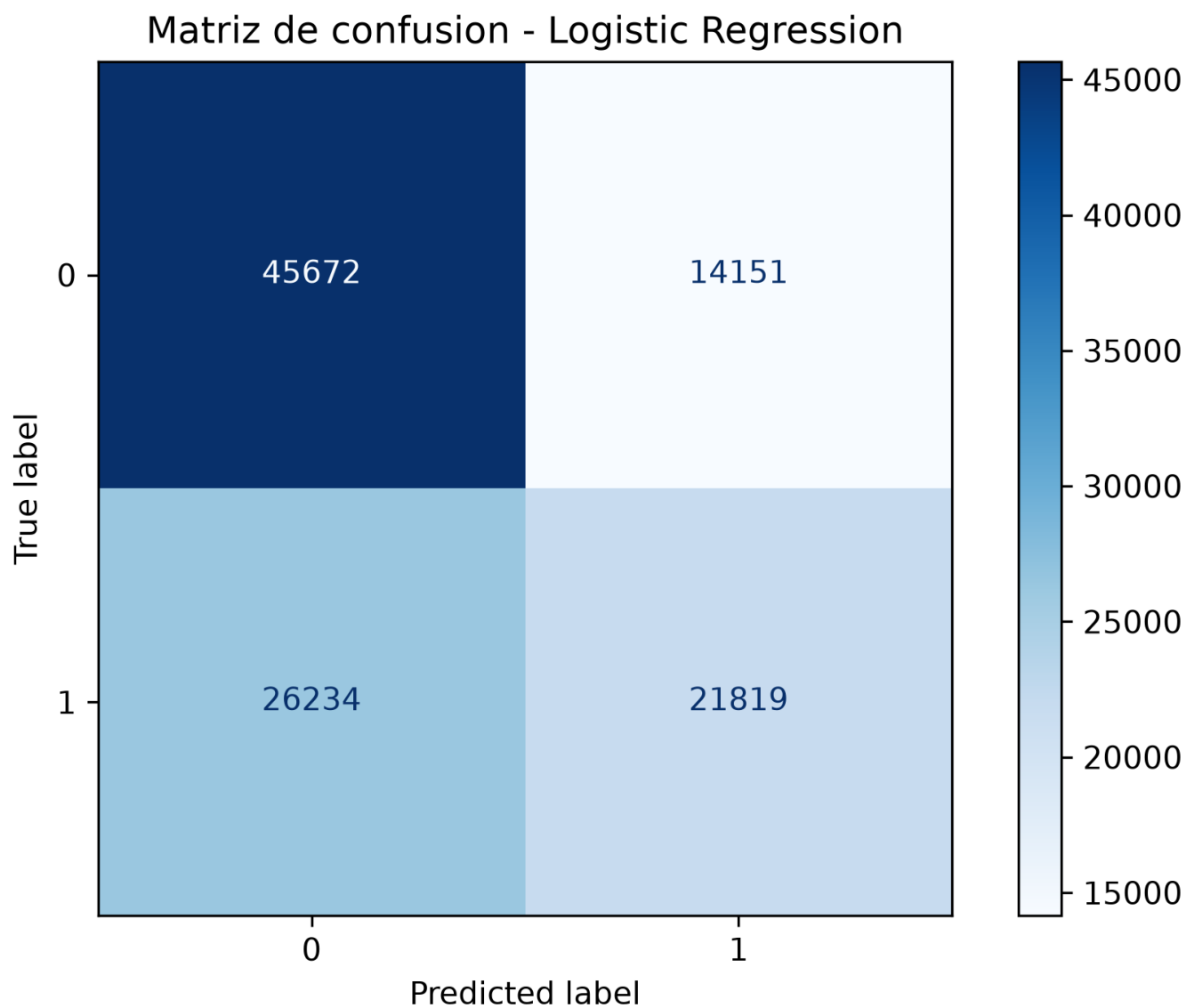
Mediciones al 25 %, 50 %, 75 % y 100 % del dataset en ambas tecnologías.

9. Comparación de modelos



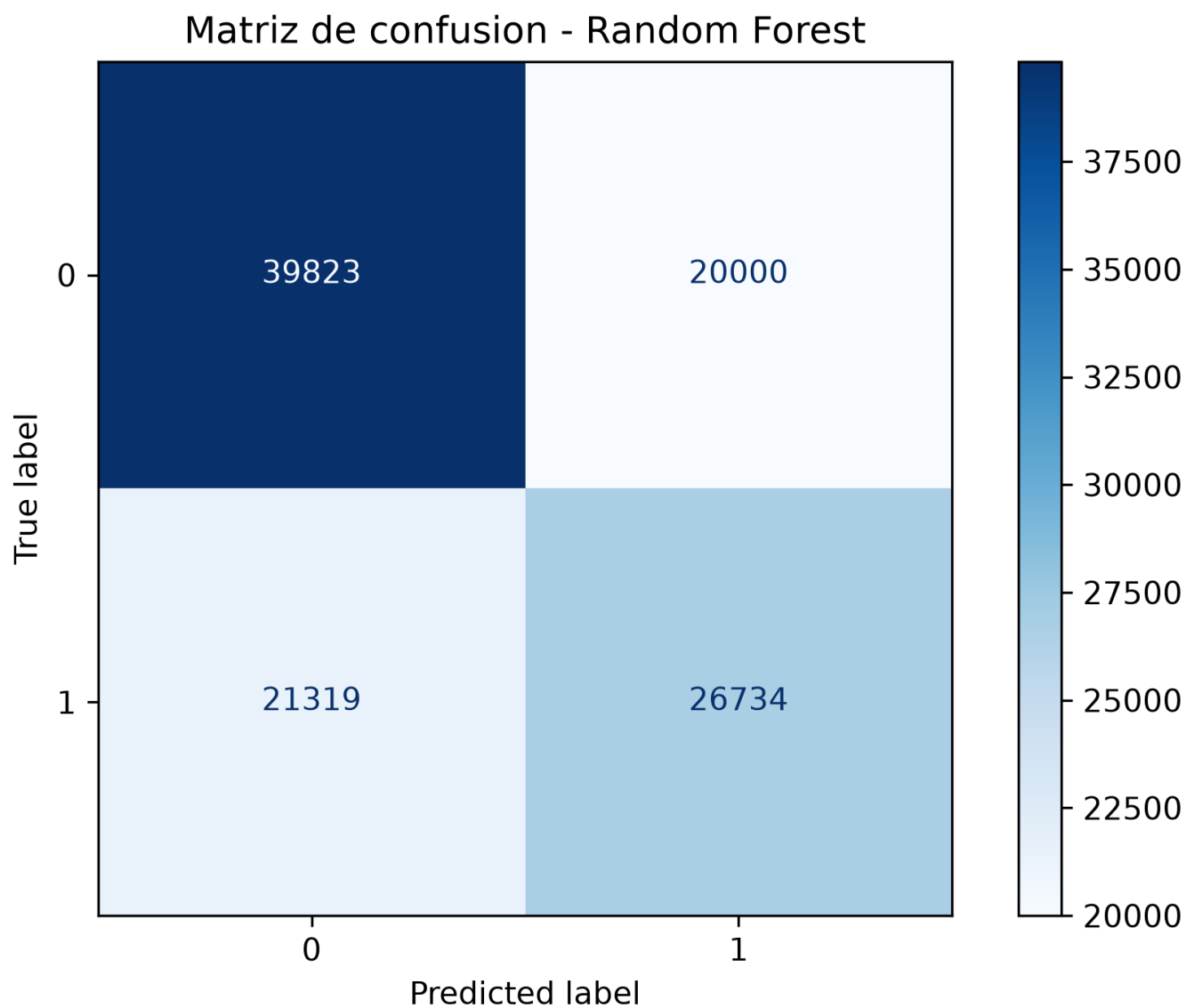
Accuracy, F1 weighted, AUC y tiempo de entrenamiento para los tres modelos.

10. Logistic Regression

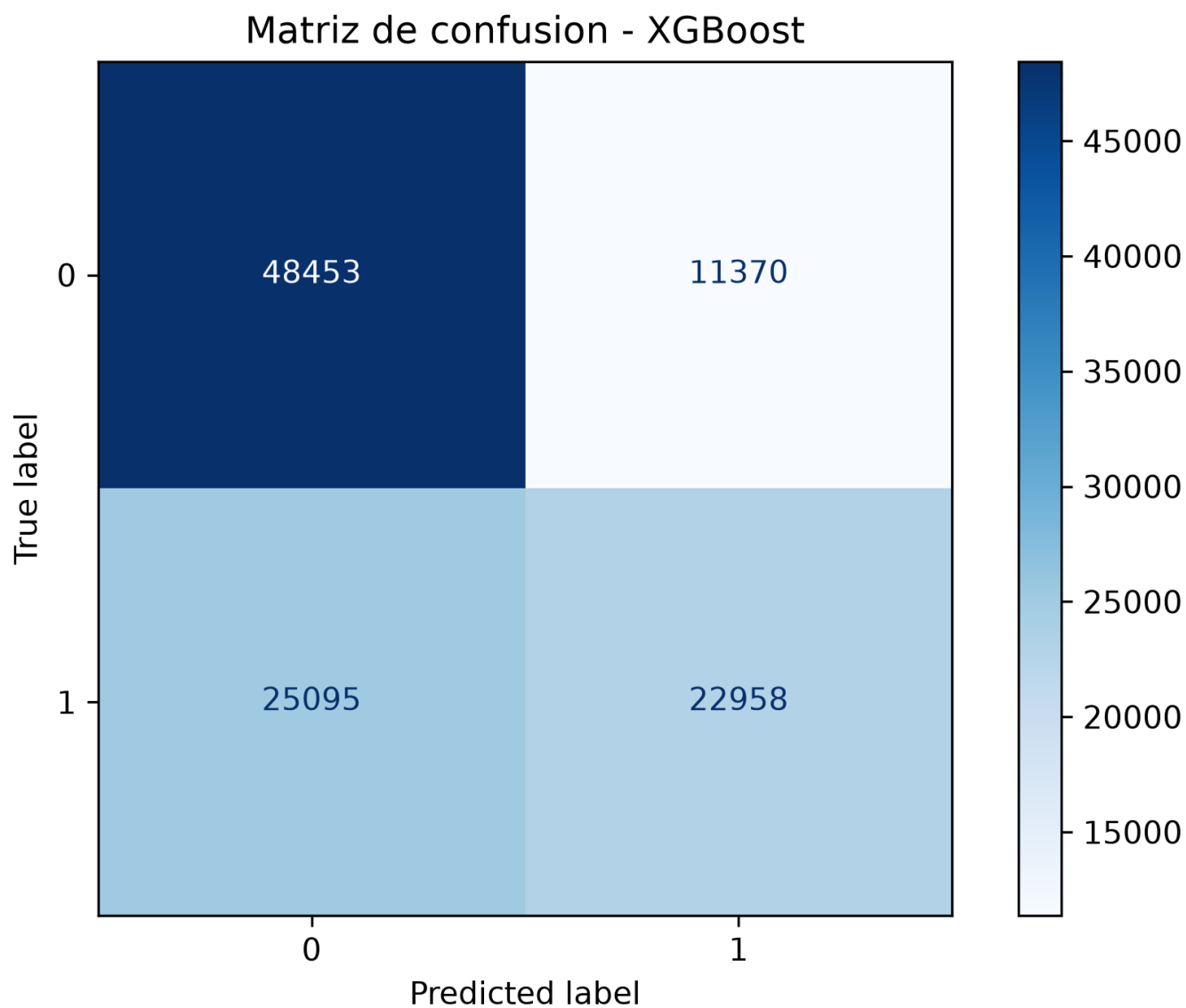


Matriz de confusión del modelo lineal utilizado como línea base.

11. Random Forest



12. XGBoost



Matriz de confusión del modelo con mejor AUC y F1 weighted.

13. Respuestas de análisis

1. Ventajas de Polars

Redujo el tiempo en todas las operaciones. La API de expresiones permitió encadenar transformaciones vectorizadas sin ciclos por fila.

2. Mayor speedup

La lectura obtuvo 29.03x, seguida por el join con 8.17x y la ingeniería de características con 7.41x.

3. Diferencia menor

El filtrado presentó la menor aceleración, 2.39x, aunque Polars todavía fue más rápido.

4. Lazy Execution

Fue marginalmente más rápido, pero incrementó el pico de memoria. El beneficio depende de filtros, proyección y complejidad del plan.

5. Limitaciones de Polars

Existen cambios de API entre versiones y el paso a scikit-learn requiere convertir el resultado final a NumPy.

6. Ventajas de Pandas

Mantiene un ecosistema amplio, documentación abundante e integración directa con bibliotecas estadísticas.

7. Justificación de migración

El speedup total de 13.23x justifica evaluar Polars para pipelines repetitivos, considerando también el costo de migración.

8. Efecto del tamaño

El speedup fue 5.42x al 25 % y 9.32x al 100 %, por lo que la ventaja se sostuvo al aumentar los registros.

9. Mejor modelo

XGBoost fue superior con Accuracy 0.6620, F1 0.6512 y AUC 0.7134, lo que sugiere relaciones no lineales.

10. Recomendaciones

Medir múltiples repeticiones, evaluar lazy según el flujo real, validar temporalmente los modelos e incorporar clima y congestión.

14. Conclusiones

Polars superó a Pandas en las seis operaciones y alcanzó un speedup de 13.23x en el pipeline total. La prueba de escalabilidad confirma que la ventaja se mantiene con el dataset completo. Lazy execution fue ligeramente más rápido, aunque utilizó más memoria, por lo que su conveniencia debe medirse y no asumirse.

En predicción, XGBoost presentó el mejor balance de Accuracy, F1 weighted y AUC. Los resultados no implican que Polars reemplace universalmente a Pandas ni que XGBoost sea óptimo para cualquier escenario; respaldan su elección para este dataset y este entorno experimental.

La solución cumple el objetivo de construir pipelines equivalentes, documentar las decisiones de ingeniería, generar evidencias reproducibles y fundamentar las conclusiones con resultados medidos.

Referencias

Kaggle. Airlines Dataset to Predict a Delay.

<https://www.kaggle.com/datasets/jimschacko/airlines-dataset-to-predict-a-delay>

Polars documentation. <https://docs.pola.rs/>

Pandas documentation. <https://pandas.pydata.org/docs/>